

The Ackermann Function and the Turing Machine: The Boundary of Primitive Recursion

For readers working in formal languages, mathematical logic, and the foundations of computability.

Formal-verification layer. The Lean 4 snippets in this essay are compiled against a pinned Mathlib revision and are labelled $\square$ verified (Lean 4.32.0, Mathlib @ v4.32.0, 2026-09-02). The reproducible lake project lives in `ack-lean/` (this directory); the full source is `AckLean/AckEssay.lean`. Snippets marked `-- sketch`, `↪not` compiled are illustrative only. See the “Formalization frontier” note in §4 and the references for what is verified versus what is stated but not yet formalized.

Abstract

The Ackermann function $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ is the canonical witness to the strict inclusion

$$\{\text{primitive recursive functions}\} \subsetneq \{\text{total Turing-computable functions}\}.$$

It is total and effectively computable — a Turing machine evaluates it on every input and halts — yet it is not primitive recursive. We prove this in the form that makes the relationship with the Turing machine explicit. The argument rests on the family of Ackermann rows $A(k, \cdot)$: (i) each fixed row $A(k, \cdot)$ is primitive recursive; (ii) every primitive-recursive function is dominated by one *fixed* row — the **domination theorem**, proved by structural induction on the primitive-recursive construction using a single **composition bound** $A(x, A(y, z)) < A(x + y + 2, z)$, exactly as in the classical proof (cf. ProofWiki; a machine-checked version is in the Isabelle AFP); and (iii) the diagonal $n \mapsto A(n, n)$ has its row-index equal to its input, so it escapes every fixed row and is therefore not primitive recursive. The resource corollary is the operational content: **no Turing machine computing A has a primitive-recursive bound on its running time**, by the exact machine-model characterization $f \in \text{PR} \iff \text{some Turing machine computes } f \text{ within a primitive-recursive clock}$. We close by situating the result in proof theory: A is provably total in Peano Arithmetic, at the finite fast-growing-hierarchy level F_ω , whereas the Kirby-Paris hydra game sits at the much higher ordinal ε_0 and is *not* provably total in PA — two distinct thresholds, not one.

The machine-checked layer of this essay follows the classical proof in the form it appears in **Mathlib** (`Mathlib.Computability.Ackermann`), whose domination theorem is stated for the *unary + pairing* formulation of primitive recursion and uses a *max* composition bound. The essay’s *multivariate* proof (with the *sum* bound) is the same theorem in the textbook formulation; §5.2 gives the essay’s sum bound as a **two-line verified corollary** of Mathlib’s max bound, and §5.x makes the two formulations and their bridge explicit.

1. Thesis and roadmap

A reader might ask: *in what sense* does the Ackermann function “relate to” a Turing machine? The naive answer — “a Turing machine can compute it” — is true but vacuous, since every total computable function is Turing-computable by definition. The substantive relationship is that A is the **canonical boundary object** between two classes that a single machine model straddles:

$$\underbrace{\text{PR}}_{\text{provably PR-bounded computation}} \subsetneq \underbrace{\text{total computable}}_{\text{halts, but no PR time bound}} = \text{Turing-computable} = \mu\text{-recursive}.$$

We use “canonical” deliberately, not “least” or “immediate successor”: in the quasi-order of total functions under eventual domination there is a dense zoo of total recursive functions strictly between the PR functions and A (and even below A on some inputs), so A is not the least total-computable function escaping the PR class. What makes A canonical is that it is the *first simply-defined* function to do so — the standard textbook witness to the strictness of the inclusion, and, as we show, one for which no PR clock can certify the computation.

Concretely, following a structure in which the Ackermann rows do the work:

1. **Definition and totality.** The Ackermann-Péter function, its first rows, and a constructive termination argument (§2).
2. **Turing computability.** An explicit stack-machine, and the μ -recursive reading (§3).
3. **The machine-model characterization of PR.** $f \in \text{PR} \iff$ a Turing machine computes f within a PR time bound (§4).
4. **The Ackermann rows exhaust PR.** Each fixed row is PR; the domination theorem; a closing inequality (§5).
5. **A is not PR.** The diagonal escape (§6), and the resource corollary.
6. **Proof theory, briefly.** A is provably total in PA at level F_ω ; the hydra is at ε_0 and is not (§7).

The fast-growing hierarchy appears only at the end, as a *conceptual* comparison (§5.4), not as the engine of the proof.

2. The Ackermann-Péter function

We use the two-argument (Péter) form, the standard one in the literature on primitive recursion:

$$\begin{aligned} A(0, n) &= n + 1, \\ A(m + 1, 0) &= A(m, 1), \\ A(m + 1, n + 1) &= A(m, A(m + 1, n)). \end{aligned} \tag{1}$$

(*Historical note.* Ackermann (1928) introduced a related three-argument function in his study of a gap in Hilbert's finitist programme; Péter (1956–58) recast it as (1) and proved it is not primitive recursive. The name “Ackermann function” now refers to (1).)

In Mathlib this is the plain total `def Ackermann.ack`, whose equation compiler accepts the recursion exactly as (1):

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
import Mathlib.Computability.Ackermann
open Nat Ackermann
#check ack          -- ack : ℕ → ℕ → ℕ
#eval ack 3 4       -- 125 (= 2^(4+3) - 3)
#eval ack 4 1       -- 65533 (= ↑↑24 - 3, the first "unwritable" row)
```

2.1 The first rows: one iteration level per row

Unfolding (1) gives closed forms for the low rows. Each row is exactly one *iteration level* above the previous:

m	$A(m, n)$	operation
0	$n + 1$	successor
1	$n + 2$	addition (from 2)
2	$2n + 3$	multiplication
3	$2^{n+3} - 3$	exponentiation
4	$2 \uparrow\uparrow (n + 3) - 3$	tetration (power tower)

(2)

where $\uparrow\uparrow$ is Knuth's double arrow: $a \uparrow\uparrow 1 = a$ and $a \uparrow\uparrow (k + 1) = a^{a \uparrow\uparrow k}$.

Proof sketch (by induction on m). $m = 0$ is immediate. Suppose $A(m, n) = 2n + 3$. Then $A(m + 1, 0) = A(m, 1) = 2 \cdot 1 + 3 = 5 = 2^{1+3} - 3$, and

$$A(m + 1, n + 1) = A(m, A(m + 1, n)) = 2 A(m + 1, n) + 3 = 2(2^{n+3} - 3) + 3 = 2^{n+4} - 3,$$

so $A(m + 1, n) = 2^{n+3} - 3$. The $m = 3 \rightarrow 4$ step is identical in shape but replaces the linear recurrence $x \mapsto 2x + 3$ by $x \mapsto 2^x - 3$, yielding a power tower: $A(4, n + 1) = 2^{A(4, n)} - 3$ with $A(4, 0) = 13 = 2 \uparrow\uparrow 3 - 3$, hence $A(4, n) = 2 \uparrow\uparrow (n + 3) - 3$. $\square$

Rows 0–3 of (2) are simp lemmas in Mathlib; row 4 (tetration) is **not** in the current revision and is left to the reader:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
#check ack_one      -- ack 1 n = n + 2
#check ack_two      -- ack 2 n = 2 * n + 3
#check ack_three    -- ack 3 n = 2 ^ (n + 3) - 3
-- row 4 (tetration, ↑↑2(n+3) - 3) is not in Mathlib v4.32.0 - left to the reader.
```

The pattern is the intuitive engine of everything that follows: **each increment of the first argument adds one level of iteration** (successor $\rightarrow + \rightarrow \times \rightarrow \rightarrow \uparrow\uparrow$), which is precisely what the fast-growing hierarchy formalizes in §5.4.

The growth is not merely fast — it is *unwritable* within a few rows. By direct computation:

- $A(4, 1) = 65\,533$;
- $A(4, 2) = 2 \uparrow\uparrow 5 - 3 = 2^{65536} - 3$, an integer with **19 729** decimal digits;
- the diagonal value $a(4) = A(4, 4) = 2 \uparrow\uparrow 7 - 3$ is a power tower of seven 2's; its *decimal digit count* is itself the 19 728-digit integer $\approx 6.03 \times 10^{19\,727}$.

2.2 Totality and monotonicity

Proposition 2.1 (Totality). A is total: $A(m, n)$ is defined for all $m, n \in \mathbb{N}$.

Proof (by induction on m). $A(0, n) = n + 1$ is total. Assume $A(m, \cdot)$ is total. We show $A(m + 1, \cdot)$ is total by induction on n : $A(m + 1, 0) = A(m, 1)$ is defined by the inductive hypothesis on m ; and $A(m + 1, n + 1) = A(m, A(m + 1, n))$ is defined because $A(m + 1, n)$ is (inner induction) and $A(m, \cdot)$ is total (outer induction). $\square$

This is a genuine, constructive termination argument — not an appeal to a completeness theorem — and it is what licenses the Turing machine of §3.

The Lean contrast. In Lean, `ack` is a *plain total def*: the equation compiler accepts (1) because it synthesises a well-founded relation on the *nested* pair (m, n) (lexicographic in m , then the recursive call in n shrinks). That synthesized witness is *exactly* the relation our double induction above mirrors by hand. The checker does not “reduce totality to an oracle”: it checks the same well-founded descent, and the human proof of Prop 2.1 is the readable version of the witness the checker builds. We keep the human proof primary and note the contrast so that “the termination checker accepted it” is not mistaken for a completeness theorem.

Proposition 2.2 (Monotonicity). A is strictly increasing in each argument: $A(m, n + 1) > A(m, n)$ and $A(m + 1, n) > A(m, n)$ for all m, n .

Proof. By simultaneous induction on m , the second-variable part by an inner induction on n at each m .

Base $m = 0$. $A(0, n + 1) = n + 2 > (n + 1) = A(0, n)$; and $A(1, n) = n + 2 > (n + 1) = A(0, n)$.

Inductive step. Assume both parts hold for m (so in particular $A(m, \cdot)$ and $A(m + 1, \cdot)$ are strictly increasing in their second argument).

- *Second variable.* $A(m + 1, n + 1) = A(m, A(m + 1, n))$ and $A(m + 1, n) = A(m, A(m + 1, n - 1))$ (with $A(m + 1, 0) = A(m, 1) > A(m, 0)$ the $n = 0$ case). By the inner induction $A(m + 1, n) > A(m + 1, n - 1)$, and since $A(m, \cdot)$ is strictly increasing, $A(m, A(m + 1, n)) > A(m, A(m + 1, n - 1))$, i.e. $A(m + 1, n + 1) > A(m + 1, n)$.
- *First variable.* For $n = 0$: $A(m + 2, 0) = A(m + 1, 1) > A(m + 1, 0)$ since $A(m + 1, \cdot)$ is strictly increasing. For $n \geq 1$: $A(m + 2, n) = A(m + 1, A(m + 2, n - 1))$ and, by the inner induction on n , $A(m + 2, n - 1) > A(m + 1, n - 1)$; applying the strictly-increasing map $A(m + 1, \cdot)$ and using $A(m + 1, A(m + 1, n - 1)) = A(m + 1, n)$ gives

$$A(m + 2, n) = A(m + 1, A(m + 2, n - 1)) > A(m + 1, A(m + 1, n - 1)) = A(m + 1, n).$$

□

The proof is a simultaneous double induction: the first-variable step at $(m + 1, n)$ is fed by the first-variable step at $(m + 1, n - 1)$ (inner induction) composed with the second-variable monotonicity of $A(m + 1, \cdot)$. Both statements are lemmas in Mathlib:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
#check ack_strictMono_right -- ∀ m, StrictMono (ack m)           (2nd
  ↪argument)
#check ack_strictMono_left  -- ∀ n, StrictMono (fun m => ack m n) (1st
  ↪argument)
```

3. A is Turing-computable

3.1 The machine

We describe a deterministic one-tape Turing machine M_A that, on input the pair (m, n) (in any fixed standard encoding), halts with $A(m, n)$. The tape plays two roles: a **stack of pending frames** (m', n') and a **work area** for the current value.

M_A pushes the frame (m, n) and loops:

1. Pop the top frame (m', n') .
2. If $m' = 0$: the value is $n' + 1$; deliver it (to the continuation frame below, or as the final output).
3. If $n' = 0$: push $(m' - 1, 1)$ [the clause $A(m + 1, 0) = A(m, 1)$].
4. If $n' > 0$: push a *continuation* frame $(m' - 1, \cdot)$ and then push $(m', n' - 1)$. The machine first computes $A(m', n' - 1)$; when that value v is returned, it pushes $(m' - 1, v)$, thereby computing $A(m' - 1, v) = A(m', n')$ [the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$].

Because A is total (§2.2), the stack is always eventually exhausted, so M_A halts on every input. Finite control plus a tape-as-stack is a legitimate Turing machine, so A is **Turing-computable**.

Mathlib's route. Mathlib does *not* formalise the stack machine M_A above. It proves ack computable via a **partial-recursive code** (`Nat.Partrec.Code.pappAck`)

and the theorem `computable2_ack : Computable2 ack`, with the Partrec-to-Turing-machine bridge `PartrecToTM2.tr_eval`. M_A is therefore the *conceptual* machine of the essay; the machine-checked witness that A is Turing-computable is `computable2_ack`.

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
import Mathlib.Computability.TuringMachine.ToPartrec
#check 2computable_ack      -- 2Computable ack (via the Partrec code pappAck)
#check PartrecToTM2.tr_eval -- eval (TM2.step tr) (init c v) = halt <$>
    ↪ Code.eval c v
```

3.2 The μ -recursive reading

Equivalently, A is a **general (μ -)recursive function**. The recursion (1) is *not* a primitive recursion: in the clause $A(m+1, n+1) = A(m, A(m+1, n))$ the argument of the outer call is itself the value of a recursive sub-computation, so the *depth* of the recursion on an input (m, n) is a function of the *computed values*, not a fixed primitive-recursive function of the input. This is a syntactic observation about the *shape* of the definition — the recursion is *general* (its unfolding depth is data-dependent) rather than *primitive* (whose unfolding depth is bounded a priori). It is, however, a *well-founded* definition: §2.2 proves by double induction that every sub-computation is eventually evaluated, so the recursion terminates on every input. By the fundamental equivalences of the 1930s —

$$\text{general recursive} = \mu\text{-recursive} = \text{Turing-computable} = \lambda\text{-definable}$$

(Kleene 1936; Church 1936; Turing 1936) — A is computed by a Turing machine. The point of §6 is that A lies in this class **but not** in the primitive-recursive sub-fragment; in particular, the data-dependent depth is *not* bounded by any primitive-recursive function of (m, n) (Corollary 6.2), which is the resource-theoretic content of the main theorem.

4. Primitive recursion and its exact Turing-machine characterization

A function is **primitive recursive (PR)** iff it belongs to the smallest class containing the zero function, the successor $S(x) = x + 1$, and the projections $\pi_i^k(x_1, \dots, x_k) = x_i$, and closed under **composition** and **primitive recursion**:

$$f(x, 0) = g(x), \quad f(x, t + 1) = h(x, t, f(x, t)) \implies f \text{ is PR if } g, h \text{ are.}$$

The bridge between the syntactic class “PR” and the operational class “Turing machine with a PR clock” is the following.

Theorem 4.1 (PR = PR-bounded-time Turing-computable). A function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is primitive recursive iff there exist a deterministic one-tape Turing machine M computing f and a primitive-recursive function $t : \mathbb{N} \rightarrow \mathbb{N}$ such that M halts within $t(|x|)$ steps on every input x .

This is folklore/textbook material; a recent paper gives it a fully explicit machine-model form (Schwartz 2025). We prove it.

Proof.

($\Rightarrow$) By structural induction on f . The initial functions are computed by trivial machines with obvious (linear, hence PR) step bounds. Composition: run the machines for the sub-functions in sequence; the step bound is the sum (a PR function) of the sub-bounds. Primitive recursion: $f(x, t)$ is computed by a machine that simulates the step h exactly t times; since t is part of the input and h has a PR step bound, the total bound is PR (a

PR-bounded loop). Every PR function is thus computed by a machine whose running time is PR-bounded.

($\Leftarrow$) Conversely, suppose M computes f and halts within $t(|x|)$ steps with t PR. By the standard Gödel coding, the configuration of a one-tape machine at any instant — (state, head position, finite non-blank tape contents, the two head-adjacent symbols) — is encoded as a single natural number, under which coding the one-step transition $\text{cfg} \mapsto T(\text{cfg})$ is **primitive recursive** (indeed Δ_0), being a finite case analysis over the finite transition table. Make the halting states *absorbing* (T fixes a halting configuration) so the s -fold iterate is defined for all s , and define the iterate **explicitly by primitive recursion**:

$$g(y, 0) = y, \quad g(y, s + 1) = T(g(y, s)).$$

Since T is PR, g is PR in (y, s) . The configuration after s steps is $\text{cfg}_s = g(\text{cfg}_0, s)$, hence PR in (cfg_0, s) — the content of “bounded iteration of a PR function is PR.” Taking $s = t(|x|)$ (a PR value) and reading the output from $\text{cfg}_{t(|x|)}$ (which equals the halting configuration, since M halted by then) yields f as a PR function. $\square$

Corollary 4.2. f is *not* primitive recursive iff **no** Turing machine computing f has a primitive-recursive time bound.

This corollary is the resource-theoretic translation of everything that follows: once we prove A is not PR, it is automatic that *no* Turing machine computing A (including M_A of §3) has a PR running-time bound. We are careful about the precise meaning: “no PR time bound” means there is no PR function t that upper-bounds the running time of *every* machine computing A . It does *not* assert that the running time of a given machine eventually dominates every PR function — that stronger claim is not a formal consequence (see the remark after Corollary 6.2).

Formalization frontier (Theorem 4.1, the $\Leftarrow$ direction). What Mathlib v4.32.0 already provides is the *Partrec $\Rightarrow$ Turing machine* half: `PartrecToTM2.tr_eval` shows every partial-recursive code is evaluated by a concrete TM2 machine, in polynomial time, and the `Turing.TM2ComputableInTime` structure is the machine + PR-time-bound model. What is **not** a single existing Mathlib theorem is the $\Leftarrow$ direction in the *PR-clock* form — “a TM computes f within a PR time bound $t \Rightarrow f$ is `Nat.Primrec`” via Gödel-coding of configurations, PR-ness of the one-step transition, and bounded iteration. We state it here as the frontier item **B(iii)** (see the “Formalization frontier” note at the end) rather than claiming it as verified.

5. The Ackermann rows exhaust the primitive-recursive functions

The scale that does the work in the proof is the family of Ackermann rows $A(k, \cdot)$. We first note that each fixed row is itself PR, then prove the domination theorem.

5.1 Each fixed row is primitive recursive

Proposition 5.1. For each fixed $k \in \mathbb{N}$, the one-variable function $n \mapsto A(k, n)$ is primitive recursive.

Proof (by induction on k). $A(0, n) = n + 1 = S(n)$ is PR. Suppose $A(k, \cdot)$ is PR. The row $A(k + 1, \cdot)$ is defined by

$$A(k + 1, 0) = A(k, 1) \quad (\text{a constant, hence PR}), \quad A(k + 1, n + 1) = A(k, A(k + 1, n)).$$

The second clause is a **genuine primitive recursion** in n , with the PR function $z \mapsto A(k, z)$ (by the inductive hypothesis) as the step function. Hence $A(k + 1, \cdot)$ is PR. $\square$

Two points deserve emphasis. First, the proof is by induction on the *fixed* parameter k : it produces, for each numeral k , a *separate* PR definition of the row $A(k, \cdot)$. Second —

and this is the phenomenon the whole essay turns on — it does **not** produce a single PR function $\Phi(k, n) = A(k, n)$ of *two* variables. The uniform two-variable function is *not* primitive recursive; if it were, its diagonal $n \mapsto A(n, n)$ would be PR (the diagonal of a PR function is PR), contradicting Theorem 6.1. “Every fixed row is PR, but the family is not uniformly PR in its row-index” is the Ackermann phenomenon in miniature.

The induction on k is a short Lean proof:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
-- For each fixed  $k$ ,  $n \mapsto \text{ack } k \ n$  is primitive recursive. The induction on  $k$ 
  ↳ produces,
-- for each numeral  $k$ , a separate PR proof – it does NOT produce a single PR
  ↳ function
--  $(k, n) \mapsto \text{ack } k \ n$  (that is exactly what Theorem 6.1 denies, via the diagonal).
theorem ack_row_primrec (k : ℕ) : Nat.Primrec (fun n => ack k n) := by
  induction k with
  | zero =>
    exact Nat.Primrec.succ.of_eq (fun n => by rw [ack_zero])
  | succ k IH =>
    have hstep : ₂Primrec (fun _ (v : ℕ) => ack k v) :=
      ₂Primrec.mk (Nat.Primrec.comp IH Primrec.snd)
    exact (Primrec.₁nat_rec (ack k 1) (fun _ (v : ℕ) => ack k v) hstep).of_eq fun n
      ↳ => by
        induction n with
        | zero => rw [ack_succ_zero]
        | succ n IHn => rw [Nat.rec_succ, ack_succ_succ, IHn]
#check ack_row_primrec 1
#check ack_row_primrec 3
```

5.2 The composition bound

The domination theorem is proved by structural induction on the primitive-recursive construction. The only non-obvious step is the primitive-recursion case, and it closes with a single **composition bound** — the one inequality the whole argument turns on.

Lemma 5.2 (Composition bound). For all $x, y, z \in \mathbb{N}$,

$$A(x, A(y, z)) < A(x + y + 2, z). \quad (3)$$

Proof. From $x + y + 1 > y$ and monotonicity in the first argument (Proposition 2.2), we have $A(x + y + 1, z) > A(y, z)$. Now

$$A(x + y + 2, z) \geq A(x + y + 1, z + 1) =$$

$$A(x + y, A(x + y + 1, z)) > A(x + y, A(y, z)) \geq A(x, A(y, z)),$$

where the equality is the recursion (1) (the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$ with $m = x + y$, $n = z$), the first inequality is monotonicity in the second argument, and the last two are monotonicity in the first argument together with $A(x + y + 1, z) > A(y, z)$. $\square$

The proof is a one-line chain; it is the exact form used in the classical domination proof (cf. ProofWiki, “Strict Upper Bound for Composition of Ackermann-Péter Functions”). Intuition: composing two rows costs at most the *sum* of their indices plus two — a row of Ackermann is a *level* of iteration, and levels add under composition, with the argument z passed through untouched.

The essay's sum bound, verified. Mathlib's lemma is the *max* form; the essay's sum form (3) is a two-line corollary of it, and is proved as a named lemma in the Lean layer:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
-- Mathlib's actual lemma (the MAX form):
```

```

#check ack_ack_lt_ack_max_add_two -- ack m (ack n k) < ack (max m n + 2) k
-- The essay's SUM form (3), a two-line corollary: max x y ≤ x + y, and ack · z is
-- monotone in its first argument. This is the plan's B(i) – the essay's *specific*
-- bound, now a verified named lemma.
theorem ack_comp_bound_sum (x y z : ℕ) : ack x (ack y z) < ack (x + y + 2) z := by
  calc
    ack x (ack y z) < ack (max x y + 2) z := ack_ack_lt_ack_max_add_two x y z
    _ ≤ ack (x + y + 2) z :=
      ack_mono_left z (Nat.add_le_add (max_le.mpr (Nat.le_add_right x y,
        ↪ Nat.le_add_left y x) (le_refl 2)))

```

Remark (relation to the base closing). A weaker but sometimes-convenient corollary of (3) is the *base closing* $A(b, A(K, n)) \leq A(K, n + 1)$ for $b < K$ (take $x = b, y = K - b, z = n$ in a rearranged form, or use the identity $A(K, n + 1) = A(K - 1, A(K, n))$ directly). We do not need it below; the composition bound (3) suffices.

5.3 The domination theorem

Theorem 5.3 (Domination). Let $f : \mathbb{N}^k \rightarrow \mathbb{N}$ be primitive recursive. Then there is a fixed integer t_f (depending only on f) such that

$$f(x_1, \dots, x_k) < A(t_f, \max\{x_1, \dots, x_k\}) \quad \text{for all } x. \quad (4)$$

Equivalently, f is dominated by one *fixed* row $A(t_f, \cdot)$.

Proof (structural induction on the construction of f). We assign to each PR function a **level** t and prove the bound (4) at that level. The level is a *per-function constant* that is **level-additive** under composition and recursion: it grows by a fixed amount at each construction step, so it stays a fixed integer depending only on f .

- **Initial functions.** The zero function and the projections have level $t = 0$: $0 \leq x_j \leq \max x < A(0, \max x)$ (since $A(0, m) = m + 1$). The successor has level $t = 1$: $S(x) = x + 1 < x + 2 = A(1, x) = A(1, \max\{x\})$. (A constant c has level $t = c$: $c \leq A(c, \max x)$ for $\max x \geq 1$, and the $\max x = 0$ case is absorbed by raising t by one.)
- **Composition.** Let $f = h(g_1, \dots, g_r)$ with g_i at level t_i and h at level t_h by induction. Put $t_g = \max_i t_i$ and $t_f = t_h + t_g + 2$. By monotonicity in the first argument, $g_i(x) \leq A(t_g, \max x)$ for all i , so $\max_i g_i(x) \leq A(t_g, \max x)$. By monotonicity of h and the induction hypothesis on h ,

$$f(x) = h(g_1(x), \dots, g_r(x)) <$$

$$A(t_h, \max_i g_i(x)) \leq A(t_h, A(t_g, \max x)) < A(t_h + t_g + 2, \max x) = A(t_f, \max x),$$

where the third inequality is the composition bound (3) with $x = t_h, y = t_g, z = \max x$.

- **Primitive recursion.** Let $f(x, 0) = g(x), f(x, n + 1) = h(x, n, f(x, n))$, with g at level t_g and h at level t_h by induction. Write $m = \max\{x_1, \dots, x_k\}$ and set $t_f = \max(t_g, t_h) + 1$. We prove

$$f(x, n) < A(t_f, m + n) \quad \text{for all } n \quad (5)$$

by induction on n .

Base $n = 0$. $f(x, 0) = g(x) < A(t_g, m) \leq A(t_f, m) = A(t_f, m + 0)$, since $t_g \leq t_f$ and $A(t_f, \cdot)$ is increasing in its first argument.

Step. Assume (5) at n . Then $f(x, n) < A(t_f, m + n)$, and

$$f(x, n + 1) = h(x, n, f(x, n)) < A(t_h, \max\{m, n, f(x, n)\}).$$

We claim $\max\{m, n, f(x, n)\} \leq A(t_f, m + n)$. Indeed: $m \leq A(t_f, m + n)$ and $n \leq A(t_f, m + n)$ (both follow from $A(t_f, w) \geq w$ for $w \geq 1$, with the $w = 0$ case absorbed by raising t_f), and $f(x, n) < A(t_f, m + n)$ by the induction hypothesis. Hence

$$f(x, n + 1) < A(t_h, A(t_f, m + n)) \leq A(t_f, A(t_f, m + n)) = A(t_f, m + n + 1),$$

where the inequality is monotonicity in the first argument ($t_h \leq t_f$) and the equality is the recursion (1) (the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$ with $m = t_f$, $n = m + n$). This is (5) at $n + 1$, completing the induction.

Since every PR function is built from the initial functions by finitely many compositions and primitive recursions, the induction assigns to each one a fixed level t_f and establishes (4). $\square$

Remark (why the level stays fixed). The primitive-recursion step is where a naive proof fails: iterating the step map n times seems to accumulate a growing amount into the row index. The trick, which is exactly what the classical proof does, is to hold the *level* $t_f = \max(t_g, t_h) + 1$ **fixed** during the induction on n and let the *argument* be $m + n$ — the input size plus the recursion depth. The claim $\max\{m, n, f(x, n)\} \leq A(t_f, m + n)$ is what makes the step close: the recursion depth n and the previous value $f(x, n)$ are both absorbed into the *argument* $A(t_f, m + n)$, not into the level. The level is therefore a fixed per-function constant, and (4) follows. This is the same level-additivity that the composition bound (3) encodes: a fixed level is preserved under one step of primitive recursion, with the work of the step absorbed into the argument rather than the row.

Theorem 5.3 is the content: the Ackermann rows *exhaust* the primitive-recursive functions. No matter how a PR function is built — however many nested primitive recursions — it is bounded by one *fixed* row $A(t_f, \cdot)$. Mathlib states the same content for the unary formulation:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
#check exists_lt_ack_of_nat_primrec
-- {f : ℕ → ℕ} (hf : Nat.Primrec f) : ∃ m, ∀ n, f n < ack m n
```

5.x The same theorem, machine-checked (the two-formulation spine)

The essay's Theorem 5.3 is the **multivariate** textbook proof: it works with k -ary PR functions, the **sum** composition bound (3), and a $+1$ level at the recursion step. Mathlib's `exists_lt_ack_of_nat_primrec` is the **same theorem in the unary + pairing formulation**: its PR inductive is the unary `Nat.Primrec` (constructors `zero`, `succ`, `left`, `right`, `pair`, `↪comp`, `prec`), its composition bound is the **max** form `ack_ack_lt_ack_max_add_two`, and its recursion step lands at the constant `max a b + 9` (via the pairing helpers `ack_pair_lt` and `ack_add_one_sq_lt_ack_add_three`). The two are the same class, and the bridge is `Nat.Primrec2`:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
-- A binary f is primitive recursive iff its curried version is Nat.Primrec:
#check >_Primrec
#check >_Primrec.mk      -- {f : α → β → σ} (hf : Primrec (fun p => f p.1 p.2)) :
  ↪>_Primrec f
```

The essay's sum bound (3) is not a different theorem from Mathlib's max bound — §5.2 derives it in two lines from `ack_ack_lt_ack_max_add_two` + `ack_strictMono_left` (verified as `ack_comp_bound_sum`). The recursion-step constants differ ($+1$ vs $+9$) purely because the

pairing formulation pays a fixed overhead for encoding a pair as a single natural; both are *fixed per-function* constants, which is all Theorem 5.3 requires. This is the “two formulations, one class” point made checkable.

Note on the bridge name. Some references name the bridge `Nat.Primrec' / Nat.Primrec'.prim_iff`. That definition **does not exist in Mathlib v4.32.0** (it appears only as a forward-reference doc comment in `Primrec/Basic.lean`). The working bridge in this revision is `Nat.Primrec2 + Primrec2.mk`, used above.

5.4 The fast-growing hierarchy, as a conceptual comparison

The finite levels of the **fast-growing hierarchy** $(F_m)_{m \in \mathbb{N}}$,

$$F_0(n) = n + 1, \quad F_{m+1}(n) = F_m^{o n}(n), \quad (6)$$

provide an *equivalent* exhaustion scale. Each fixed level F_m is PR (a finite construction — again a metatheoretic family of separate PR definitions, not a uniform PR function of (m, n)), and every PR function is dominated by some fixed F_m (the standard majorization theorem; see Hájek & Pudlák). Unfolding (6): $F_0(n) = n + 1$, $F_1(n) = 2n$, $F_2(n) = 2^n n$ (exponential times linear, *not* quadratic), $F_3(n) \approx 2 \uparrow n$ (tetration), and $F_m(n)$ for $m \geq 4$ is a power tower whose height is $\sim F_{m-1}(n)$.

The Ackermann rows and the FGH finite levels dominate each other: each fixed row $A(k, \cdot)$ is PR (Proposition 5.1), hence is dominated by some fixed F_m ; and each F_m is PR, hence by Theorem 5.3 is dominated by some fixed row $A(k, \cdot)$. They are therefore equivalent as *scales*, and we have used the Ackermann-row form because the diagonal escape (Theorem 6.1) is cleanest there.

Diagonalizing *through* the finite levels — $n \mapsto F_n(n)$ — yields Ackermannian growth: the diagonal’s level-index equals its input, so it climbs the hierarchy without bound, sitting at the transfinite level F_ω (just above all finite F_m). This is the same diagonal-escape mechanism as Theorem 6.1, and it is the conceptual location of the Ackermann function: **at F_ω , the first level above the PR-exhausting finite levels.**

Remark (why pointwise FGH bounds are the wrong tool). It is tempting to bound $A(m, n)$ pointwise between two FGH levels and let $m = n$. The *lower* bound is false: with $F_2(n) = 2^n n$, one has $A(3, n) = 2^{n+3} - 3$ and $A(3, 8) = 2045 < 2048 = F_2(8)$, so $A(3, \cdot)$ is not eventually $\geq F_2$. The row $A(3, \cdot)$ and the level F_2 have the same *iterative* growth rate but different *constants*, and pointwise dominance fails. The diagonal-escape proof of Theorem 6.1 avoids this pitfall entirely: it needs only the *row-index* behavior (Theorem 5.3) and strict monotonicity (Proposition 2.2), not pointwise $A(m, n)$ vs $F_m(n)$ comparisons.

6. A is not primitive recursive

We are now ready for the main theorem. The argument is a **diagonal escape**: the diagonal $n \mapsto A(n, n)$ climbs the Ackermann rows without bound, while Theorem 5.3 says no primitive-recursive function can.

Theorem 6.1. The Ackermann function $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ is total and Turing-computable, but is **not** primitive recursive.

Proof. Totality and Turing-computability are §2.2 and §3. For non-PR-ness, consider the **diagonal** $a(n) = A(n, n)$.

Assume, for contradiction, that A is PR. Then $a(n) = A(n, n)$ is PR (the diagonal of a PR function is PR — it is the composition $n \mapsto A(n, n)$). By Theorem 5.3 (with $k = 1$, so $\max\{n\} = n$), there is a fixed K such that

$$a(n) = A(n, n) < A(K, n) \quad \text{for all } n. \quad (7)$$

But by Proposition 2.2, A is strictly increasing in its first argument, so $A(n, n) > A(K, n)$ for every $n > K$. This contradicts (7) for all $n > K$. $\square$

The contradiction is the exact mirror image of Theorem 5.3: Theorem 5.3 pins any PR function to one *fixed* row $A(K, \cdot)$, whereas the diagonal $a(n) = A(n, n)$ has its *row index* equal to its input, so it outruns every fixed row as n grows. Mathlib states both the diagonal and the two-argument forms:

```
-- ✓ verified (Lean 4.32.0, Mathlib @ v4.32.0)
#check not_nat_primrec_ack_self  -- ¬ Nat.Primrec (fun n => ack n n)  (the
  ↪diagonal)
#check ¬not_primrec_ack          -- ¬ ¬Primrec ack                    (the 2-arg
  ↪function)
-- The contradiction, reproduced (the heart of Theorem 6.1). In the unary
  ↪formulation
-- the domination bound (7) is strict, so at  $n = K$  it reads  $A(K, K) < A(K, K)$  – an
-- immediate  $a < a$ . (In the multivariate formulation the same contradiction is seen
-- for any  $n > K$  by monotonicity in the first argument, as in the prose above.)
theorem not_primrec_ack_essay_form : ¬ Nat.Primrec (fun n => ack n n) := fun h => by
  obtain (K, )hK := exists_lt_ack_of_nat_primrec h
  exact (hK K).false
```

Corollary 6.2 (Resource form). No Turing machine computing A has a primitive-recursive bound on its running time.

Proof. Immediate from Theorem 6.1 and Corollary 4.2. $\square$

This is the precise sense in which the Ackermann function “outruns” the primitive-recursive world: it is not that some particular machine is slow, but that **the function itself** cannot be certified by any PR clock.

Remark (the precise scope of “no PR time bound”). Corollary 6.2 asserts the *non-existence* of a PR function t upper-bounding the running time of every machine computing A . It does **not** assert that the running time of a *given* machine computing A eventually dominates every PR function. The latter would require that a particular machine's runtime $r(n)$ satisfy $r(n) > p(n)$ eventually for every PR p , which is a strictly stronger, machine-specific claim and is not a formal consequence of $A \notin \text{PR}$. What *is* true, and is the right operational statement, is that for *every* PR t there is *some* machine computing A (indeed, for large enough inputs, the natural machine M_A) whose running time exceeds t . The diagonal-escape is a statement about the *function's* growth, not about any single machine's asymptotics.

7. Proof-theoretic significance (briefly)

The PR/total-computable divide has a sharp proof-theoretic shadow. We state it briefly and carefully, because two distinct thresholds are often conflated.

7.1 Provable totality of A

- Every **primitive-recursive** function is provably total in very weak arithmetic (already in fragments with only Δ_0 or Σ_1 induction): its totality is witnessed by a straightforward induction mirroring its finite construction.
- A **is provably total in Peano Arithmetic (PA)**. The induction on m of §2.2, with the nested recursion handled by induction on n , is formalizable in PA.
- The **provably-total functions of PA** are exactly those dominated by the fast-growing hierarchy up to the ordinal ε_0 (the proof-theoretic ordinal of PA) — the classical ordinal-analysis result (Girard; Hájek & Pudlák). Concretely, f is provably total in PA iff $f(n) \leq F_\alpha(n)$ for some $\alpha < \varepsilon_0$.

The diagonal $a(n) = A(n, n)$ is dominated by the *finite* level F_ω (equivalently, by $A(n, n)$ itself, a finite row-index for each fixed input), and every finite level is $< \varepsilon_0$. Hence A is provably total in PA, sitting comfortably *inside* PA's provable totality, at the finite level F_ω — well below the ε_0 threshold. A is thus the threshold object for the *PR/total-computable* divide (provably total in PA, yet not PR), not for the *PA/true* divide.

7.2 The Kirby-Paris hydra: a different threshold

The **hydra game** (Kirby & Paris, 1982) is a finite combinatorial game whose termination is a true Π_1 statement *independent of PA*. The hydra termination-time function $H(n)$ sits at the level f_{ε_0} of the fast-growing hierarchy: H eventually dominates every f_α for $\alpha < \varepsilon_0$ (hence every function provably total in PA, and in particular every PR function and A itself), and $H < f_{\varepsilon_0+1}$. This is a *different* threshold from A 's:

object	FGH level	provably total in PA?	role
primitive recursive functions	$< F_\omega$ (finite, fixed)	yes (weak fragments)	the PR class
Ackermann $A(n, n)$	F_ω (finite, unbounded index)	yes	PR/total-computable boundary
Kirby-Paris hydra $H(n)$	f_{ε_0}	no	PA/true boundary

The hydra is *not* an Ackermann-type growth: it is Goodstein-type, at the transfinite ordinal ε_0 , and its totality is exactly what PA cannot prove. A is provably total in PA; the hydra is not. Conflating the two — e.g. claiming the hydra game “grows like $A(n, n)$ ” or that A 's totality “lands at PA's threshold” — is a scale error: A is at F_ω (inside PA), the hydra at f_{ε_0} (outside PA). Both are *boundary* objects, but for *different* boundaries.

7.3 Relation to the halting problem

The general problem “does this Turing machine halt on **all** inputs?” (“does it compute a total function?”) is undecidable — Π_2^0 -complete, strictly harder than the halting problem. The Ackermann function is the *concrete, positive* counterpart to this undecidability: it is a specific function whose totality we can *prove* (in PA) and whose machine M_A we can *write down*, yet which lies beyond the primitive-recursive, PR-bounded-computation realm. A is the boundary between the functions a weak theory can certify as total (the PR functions) and those that are total but require the full induction of PA.

8. Conclusion

The relationship between the Ackermann function and the Turing machine is not that the machine computes the function — that is shared by every total computable function. It is that **A is the canonical witness to the strictness of the inclusion**

$$\text{primitive recursive} \subsetneq \text{total Turing-computable},$$

and it realizes that strictness in three precisely-stated ways:

1. **Functionally:** A is total and Turing-computable (§3) but not primitive recursive (§6). The proof is a diagonal escape: the rows $A(k, \cdot)$ exhaust the PR functions (Theorem 5.3), and the diagonal $n \mapsto A(n, n)$ has its row-index equal to its input, so it outruns every fixed row (Theorem 6.1).
2. **Resource-wise:** by Theorem 4.1 and Corollary 6.2, A is Turing-computable but **no Turing machine computing it has a primitive-recursive time bound**. A is the first simply-defined total function a machine can evaluate that no PR clock can certify.
3. **Proof-theoretically:** A is provably total in PA at the finite level F_ω , inside PA's provable totality; the PA/true threshold is a distinct, higher object (the hydra, at f_{ε_0} , not provably total in PA).

Each increment of A 's first argument adds one iteration level — successor, addition, multiplication, exponentiation, tetration — and it is precisely this unbounded ascent through the exhaustion scale (the Ackermann rows, equivalently the finite fast-growing hierarchy), still *within* it, that places A on the boundary between what a Turing machine can compute with provably bounded effort and what it can compute at all.

(The domination proof of §5.2–5.3 follows the classical level-additivity argument, presented here in the form of ProofWiki and machine-checked in the Isabelle AFP and Mathlib4. The low-level closed forms and the composition bound of §5.2 were additionally checked by direct computation on all materializable instances; the theorems themselves are proved by induction, not by computation.)

Formalization frontier (what is verified vs. stated)

- **Verified** (compiled in ack-lean/, Mathlib v4.32.0): ack total `def`; rows 0–3 (ack_one/ack_two/ack_three); monotonicity (ack_strictMono_left/right); computable₂_ack; the **sum composition bound** ack_comp_bound_sum (B(i), §5.2); exists_lt_ack_of_nat_primrec (Thm 5.3, unary form); not_primrec₂_ack / not_nat_primrec_ack_self (Thm 6.1); the two-formulation bridge (Nat.Primrec₂/Primrec₂.mk, §5.x).
- **Frontier, not yet formalized:**
 - **B(ii)** — the *multivariate* domination theorem in the essay's exact form (sum bound, +1 level) via Nat.Primrec₂. Mathlib has the unary+pairing version; the multivariate port is the essay's own proof made formal.
 - **B(iii)** — **Theorem 4.1's $\Leftarrow$ direction** ("TM computes f within a PR time bound $t \Rightarrow f$ is Nat.Primrec") via Gödel-coding of TM configurations, PR-ness of the one-step transition, and bounded iteration. Mathlib has the ingredients (Turing.TM2ComputableInTime, config encoding, PartrecToTM2.tr_eval) but not this theorem as a single statement.

References

- **Ackermann, J.** (1928). *Zum Hilbertschen Aufbau der reellen Zahlen*. Mathematische Annalen **107**, 459–488.
- **Péter, R.** (1956–58). *Ein kombinatorischer Satz über die Funktionenklasse der primitiv-rekursiven Funktionen*. Acta Physico-Mathematica **6**, 1–10; see also *Die Theorie der rekursiven Funktionen* (Akadémiai Kiadó, 1956).
- **Goodstein, R. L.** (1947). *On the computable functions*. Journal of Symbolic Logic **12**, 127–131.
- **Kleene, S. C.** (1936). *General form of recursive function*. American Journal of Mathematics **57**, 254–263.
- **Church, A.** (1936). *An unsolvability problem in arithmetic*. Journal of the London Mathematical Society **11**, 218–228.
- **Turing, A. M.** (1936). *On computable numbers, with an application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society **42**, 230–265.
- **Rogers, H.** (1967). *Theory of Recursive Functions and Effective Computability*. MIT Press.

- **Cutland, N. J.** (1980). *Computability: An Introduction to Recursive Function Theory*. Cambridge University Press.
- **Schwartz, D. G.** (2025). *A Characterization of Turing Machines that Compute Primitive Recursive Functions*. arXiv:2510.18283 [cs.FL].
- **Hájek, P., & Pudlák, P.** (1993). *Metamathematics of First-Order Arithmetic*. Cambridge University Press. (Provably-total functions of PA; ordinal ε_0 ; majorization by the fast-growing hierarchy.)
- **Girard, J.-Y.** (1987). *Proof Theory and Logical Complexity*. (Ordinal analysis; provable recursion in PA.)
- **Kirby, L., & Paris, J.** (1982). *Accessible independence results for Peano arithmetic*. Bulletin of the London Mathematical Society **14**(4), 285–293. (The hydra game; f_{ε_0} .)
- **Odifreddi, P. E.** (1992). *Classical Recursion Theory*. North-Holland.
- **ProofWiki**. *Ackermann-Péter Function is not Primitive Recursive*. https://proofwiki.org/wiki/Ackermann-P%C3%A9ter_Function_is_not_Primitive_Recursive (CC BY-SA 3.0; the level-additivity domination proof and the composition bound $A(x, A(y, z)) < A(x + y + 2, z)$.)
- **Paulson, L. C.** (2022). *Ackermann's function is not primitive recursive*. Machine Logic (blog), <https://lawrencecpaulson.github.io/2022/09/07/Ackermann-not-PR-II.html>; formalized in the **Isabelle Archive of Formal Proofs**, entry “Ackermanns_not_PR,” https://isa-afp.org/entries/Ackermanns_not_PR.html (a machine-checked proof of the domination theorem and the diagonal escape, ~300 lines.)
- **Mathlib4**. Mathlib.Computability.Ackermann (Lean 4), https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/Ackermann.html (an independent machine-checked proof that the Ackermann function is not primitive recursive). **Pinned revision: v4.32.0** (commit 81a5d257), used by this essay's Lean layer. Also relevant: Mathlib.Computability.Primrec.Basic (Nat.Primrec, Nat.Primrec2, Primrec2.mk), and Mathlib.Computability.TuringMachine.ToPartrec (PartrecToTM2.tr_eval).
- **Alectryon** (interactive rendering) and **LeanInk** (required for Alectryon's Lean 4 support): <https://alectryon.lean-lang.org/>, <https://github.com/leanprover/leanink>.